

LogPress 평가설계 — 한눈 요약 + 풀이

1·2·3층 평가를 공식과 함께 · 앞장은 요약(옆에 부연), 뒷장은 자세한 풀이
누구나 이해할 수 있게 정리 · 2026 · LogPress

층별 메트릭 한눈에		
층	메트릭 (재는 잣대)	무엇을 재나
1층 정보 보존	보존율, marginal gain	원인 라인을 baseline보다 잘 남기나 (품질)
2층 효율/비용	KV memory 절감, TTFT, throughput, inference time	원인 유지하면서 얼마나 가볍고 빠른가 (비용)
3층 출력 충실도	log-probability, perplexity, KL divergence	압축본 출력이 원본과 같은가 (충실도)

1층 정보 보존 — 원인을 잘 남기나

G = 우리가 "이건 중요하다"고 미리 정한 정답 라인 전체 (그라운드 트루스). 지금은 이상 라벨. C = 압축기가 남긴 라인 전체 (이상이트 아니든 살아남은 전부). G ∩ C = 정답 라인 중 살아남은 것.	부연 C는 "남긴 이상라벨"이 아니라 "남긴 전부"다. 정답인지 따지는 건 교집합 (∩)에서 일어난다.
$\text{보존율} = \frac{ G \cap C }{ G } = \text{정답 라인 중 살아남은 비율}$	
예: 정답 100줄 중, 압축 후 80줄이 살아남음 → 보존율 = 80 / 100 = 80%.	부연 분모는 항상 정답 전체(G). "남긴 것 중 정답 비율"이 아니라 "정답 중 살아남은 비율" = 재현율(recall).

1층 평가 흐름

무엇을 하나 · 압축 후 원인 라인을 baseline보다 잘 남기는가	부연 1층은 모델 정답률과 무관한 "안전판". 품질만 본다.
무엇을 재나 · 압축률(50/70/90%)별 보존율 → 곡선	부연 한 점이 아니라 곡선. 세계 압축할수록 얼마나 떨어지나.
어떻게 재나 · 3개 층으로 나눠 보존율 비교	부연 전체 / 비-ERROR(하드케이스) / 앞쪽 쏠림. 하드케이스가 진짜 승부.
어떤 검정 · McNemar + BH 보정	부연 보존은 "살림/버림" 이진이라 짝진 검정이 맞다. (풀이는 뒷장)
Ablation(효과) · attention Top-K → +레벨 → +희귀도 → +위치(앞쪽 쏠림) → +head-aware → +output sensitivity 를 하나씩 쌓으며 각 단계의 보존율 증가분 측정	부연 어텐션 base에 로그 고유 신호(레벨·희귀도·위치)를 엮는 구조. 어떤 신호가 실제로 보존을 올리는지 확인 — 특히 +위치가 승부처(킬스위치 위치 AUC 0.35=G0). (2층 Ablation이 이 조합을 비용측에서 재사용)

marginal gain = 보존율(LogPress) - 보존율(레벨필터) 예) 90% - 88% = +2%p	
marginal gain = "레벨필터라는 공짜 기준선을 넘어, 우리가 추가로 건진 양". 우리 순수 기여.	부연 순서 주의: 우리 - 레벨필터(우리가 앞). 양수면 우리 승, 0이나 음수면 압축기 무의미.

2층 효율 / 비용 — 가볍고 빠르나

무엇을 하나 · 원인을 유지하면서 메모리·속도를 얼마나 줄이나	부연 1층이 "품질"이면 2층은 "그래서 쓸 만하냐(비용)".
무엇을 재나 · KV 메모리 절감량 / 속도(TTFT·throughput·inference time)	부연 메모리는 압축의 직접 타깃, 속도는 체감 성능.
어떻게 재나 · 압축 전 vs 후를 같은 조건(같은 GPU·배치·컨텍스트 길이)에서 실측, 여러 번 평균	부연 첫 실행(워밍업)은 버림. 절감값을 "보존한 원인 수"로 나눠 단위당으로 보고.
어떤 방법 · 실측(통계 검정 아님) + Ablation(비용측)	부연 메모리·속도는 이진 승패가 아니라 측정값 → McNemar 안 씬. 흔들림은 부트스트랩 CI로 보조.

2층 지표 뜻풀이

KV memory reduction · 압축률별 KV 캐시 감소량	부연 같은 GPU에서 처리 가능한 최대 컨텍스트 길이도 함께.
TTFT (Time To First Token) · 첫 토큰까지 시간	부연 긴 로그 읽는(prefill) 시간이 여기 다 들어감. 압축 효과가 제일 직접 드러남.
throughput · 처리량(초당 토큰 수)	부연 높을수록 좋음. KV 작아지면 ↑.
inference time · 추론 시간(전체)	부연 입력~답 끝까지 총시간.

retained evidence 단위당 절감 = $\frac{\text{절감한 메모리(또는 시간)}}{\text{보존한 원인 라인 수}}$

핵심 지표: "원인 1개 유지 하는 데 드는 메모리·시간이 얼마나". 그냥 "많이 줄임"이 아니라 원인 유지량으로 나눈 값.	부연 세계 압축하면 메모리는 당연히 줄지만 원인도 버려짐 → 원인당으로 정규화해야 공정.
--	---

주의

어텐션 추출은 forward(모델 1회 실행)가 필요해 그 자체가 비용. "evidence 단위당 순절감"으로 보고해 **압축 이득이 추출 비용을 상쇄**함을 보여야 설득력이 선다.

3층 출력 충실도 — 원본과 같게 판단하나

무엇을 하나 · 압축본과 원본의 모델 출력 분포 가 거의 같은가	부연 정답률은 프론티어도 3.9~12.5%라 둔함 → "확률·분포 차이"로 예민하게 본다.
무엇을 재나 · 정답 확신(log-prob·perplexity) / 출력 분포 차이(KL)	부연 "답이 같나"가 아니라 "확률 지형이 같나"를 본다.
어떻게 재나 · 원본(full) 넣은 출력 vs 압축본 넣은 출력을 같은 모델·프롬프트 로 비교	부연 생성 끝까지 안 해도 됨 — 정답 확률만 뽑으면 되므로 가볍다.
어떤 방법 · 분포 비교 (log-prob·perplexity·KL) + RCA는 조건	부연 제약형 RCA로 "보존이 헛되지 않다"만 최소 확인.

3층 지표 뜻풀이

출력 분포 = 모델이 답 후보마다 매긴 확률 묶음. 디스크 62% / 네트워크 25% / 메모리 9% / CPU 4% 제일 높은 걸 고르는 게 "정답". 이 확률 지형 전체 가 분포.	부연 "답이 같다"와 "분포가 같다"는 다르다. 분포가 원본과 같으면 정보 보존 ✓. 정보 없으면 확률이 뭉개짐 (다 비슷해짐).
--	---

세 지표

log-probability · 정답 토큰에 준 확률(로그). 확률 1.0 → log=0(최대), 낮을수록 음수로 커짐.	부연 왜 로그? ① 확률 곱셈이 로그 덧셈이 돼 계산 간단 ② 작은 확률차를 크게 벌려 잘 보임. 0에 가까울수록 정답 확신.
---	--

perplexity = $e^{-(\text{평균 log-probability})}$

perplexity · log-prob를 "체감 혼란도"로 되돌린 값. "몇 개 중에 헛갈리나".	부연 1이면 안 헛갈림, 10이면 10개 중 고민 수준. 낮을수록 좋음 . log-prob와 같은 걸 뒤집은 것.
---	--

$$KL(P \parallel Q) = \sum_i P(i) \cdot \log \frac{P(i)}{Q(i)}$$

KL divergence (쿨백-라이블러 발산) · 두 확률분포가 얼마나 벌어졌나. P=원본, Q=압축본. 0이면 똑같고 클수록 다름.	부연 "원본을 기준으로 압축본이 얼마나 예상을 빗나가나". 0에 가까울수록 정보 잘 보존. (log-prob=정답 하나, KL=분포 전체)
---	---

RCA는 조건

제약형(K지선다 / 라인 지목)으로 "보존이 헛되지 않다"만 최소 확인. 자유서술은 제외(오픈 모델 정답률이 바닥이라).

부록 — 자세한 풀이

앞장 요약의 "왜"와 배경. 필요할 때만 보면 된다.

A. 보존율 — 왜 "정답 중 살아남은 비율"인가

보존율의 분모가 **C(남긴 것)가 아니라 G(정답)**인 게 핵심이다. "남긴 것 중 정답 비율"(정밀도)이 아니라 **"정답 중 살아남은 비율"**(재현율)을 본다. RCA에서 중요한 건 "원인을 놓쳤나 안 놓쳤나"이기 때문이다 — 원인 하나라도 버리면 진단 불가라, 놓침(recall)이 관건이다. G는 지금 Thunderbird의 **이상(anomaly) 라벨**이라 엄밀히는 "이상 보존"이며, 진짜 "원인 보존"은 Nezh(원인 라벨)로 교차검증한다. 공식은 G가 무엇이든 그대로 성립한다.

B. 1층 검정 — McNemar와 BH를 쓰는 이유

왜 McNemar (평균 비교가 아니라)

두 압축기가 **같은 라인**을 채점하므로 두 결과가 강하게 얽힌다(쉬운 라인은 둘 다 살리고, 어려운 라인은 둘 다 버림). 이걸 독립인 척 평균으로 비교하면 분산을 잘못 추정한다. McNemar는 **한 쪽만 살린 라인(b, c)만** 보고 얽힘을 조건부로 제거한다. 2×2 에서 둘 다 보존(a)·둘 다 제거(d)는 보존율 차이식 $(a+b)/n - (a+c)/n = (b-c)/n$ 에서 애초에 상쇄되어 정보가 없다.

$$\chi^2 = \frac{(b - c)^2}{b + c} \quad (df=1) \quad \cdot \quad \text{PASS} = b > c$$

표본 작으면 연속성보정 $(|b-c|-1)^2/(b+c)$, $b+c < 25$ 면 이항 정확검정. 단위는 라인이 아니라 **버스트**(한 버스트에 원인 수백 줄이 쏠려 독립이 아니므로). 모델 1회 실행 상황에 McNemar가 정당하다 (Dietterich 1998).

왜 BH (Bonferroni가 아니라)

3층 $\times$ 4 baseline $\times$ 5 압축비 = **60번 검정**이면, 다 차이가 없어도 평균 3개(60×0.05)가 우연히 "유의"로 뜬다. Bonferroni($0.05/60 \approx 0.00083$)는 너무 빡세서 진짜 차이까지 버린다. BH는 "고른 것 중 가짜 비율(FDR)"만 5%로 유지해 **진짜 차이를 더 잘 살린다**. (엄험 대비로 Holm을 강건성 쪽으로 병기.)

C. marginal gain — 왜 "marginal"인가

경제학의 marginal(한계)은 "이미 있는 것에 한 단위 더 얻었을 때의 **추가 증가분**"을 뜻한다(marginal cost, marginal utility). 여기서도 **바탕 = 레벨필터(공짜로 깔린 것)**, 거기에 **로그 신호를 얹어 추가로 번 양**이 marginal gain이다. 절대 보존율 90%는 대부분 레벨필터 몫이라, "우리 때문에 달라진 부분(2%p)"만 떼어내려고 빼는 것이다. $gain \approx 0$ 이면 "그냥 레벨필터 쓰면 됨" = 주제 재설정 신호.

D. 2층 — 실제로 어떻게 측정하나

- **TTFT**: 입력 시각 t_0 , 첫 토큰 나온 시각 $t_1 \rightarrow TTFT = t_1 - t_0$. 압축 전/후 각각.
- **throughput**: 생성 토큰 수 $\div$ 생성 걸린 시간 = tokens/sec.
- **inference time**: 전체 end - start. 여러 번(예: 10회) 평균, 첫 실행(워밍업)은 버림.
- **KV memory**: GPU 피크 메모리 직접 측정(torch.cuda.max_memory_allocated) 또는 계산 = $2 \times \text{layers} \times \text{heads} \times \text{head_dim} \times \text{토큰수} \times \text{bytes}$.

공정 조건: 같은 GPU·배치·컨텍스트 길이에서, 무압축 vs 압축을 같은 조건으로. 그리고 위 절감값을 "보존한 원인 라인 수"로 나눠 **단위당 절감**으로 보고한다. (회의록 미결이던 "연산속도 측정법"이 이 조건 고정이다.)

E. 3층 — 출력 분포와 세 지표

출력 분포는 모델이 다음 토큰을 고를 때 각 후보에 매긴 확률의 묶음이다. 최종 답(제일 높은 것)이 같아도 분포가 다를 수 있다. 압축이 정보를 감으면 답은 같아도 확률이 **몽개진다** ($62\% \rightarrow 30\%$ 처럼 확신이 흩어짐). 정답률로는 "둘 다 디스크"라 차이 0이지만, 분포로 보면 이 몽개짐이 잡힌다 — 그래서 3층은 예민하다.

log-probability

정답 토큰에 준 확률에 로그를 씌운 값. 확률 $1.0 \rightarrow \log=0$ (최대), 낮을수록 음수로 커진다. LLM API의 `logprobs`가 바로 이것으로, "모델이 그 토큰을 얼마나 확신하고 골랐는지"를 수치로 준다 — 즉 내장 제공값을 그대로 쓰면 된다.

perplexity

$$\text{perplexity} = e^{-(1/N) \sum \log p(\text{token}_i)}$$

log-prob의 평균을 지수로 되돌려 원래 스케일로 만든 "체감 혼란도". 1이면 전혀 안 헛갈림, 클수록 헛갈림. **낮을수록 좋다**. log-prob가 높다 = perplexity가 낮다 = 안 헛갈린다 — 같은 말의 다른 표현이라 보고엔 perplexity, 계산엔 log-prob를 흔히 쓴다.

KL divergence (쿨백-라이블러 발산)

$$KL(P \parallel Q) = \sum_i P(i) \cdot \log \frac{P(i)}{Q(i)}$$

이름은 만든 두 사람(Kullback, Leibler), divergence는 "얼마나 갈라졌나". 각 후보 i 에서 원본 확률 $P(i)$ 만큼 가중해, 원본과 압축본의 확률 비율 $\log(P/Q)$ 를 더한다. 확률이 같으면 $\log(1)=0$ 이라 그 항은 0 — 즉 **다를수록 커진다**. $P=Q$ 면 0. **비대칭**이라 $KL(P \parallel Q) \neq KL(Q \parallel P)$. 원본을 반드시 기준 P 로 둔다.

지표	보는 범위	질문	좋을 때
log-prob / perplexity	정답 하나의 확률	정답을 얼마나 확신했나	log-prob $\uparrow$ / ppl $\downarrow$
KL divergence	분포 전체의 차이	출력 지형이 원본과 얼마나 다른가	KL $\rightarrow 0$

F. RCA를 "조연"으로 두는 이유

RCA 정답률은 프론티어 모델도 3.9~12.5%로 낮아 압축 A·B 차이가 노이즈에 묻힌다. 그래서 정답률을 주인공으로 쓰지 않고, 제약형(K지선다/라인 지목)으로 "보존이 헛되지 않았다"만 최소 확인한다. 진짜 판단은 1층(보존율)·3층(출력 충실도)이 맡는다.

정리 — 1층: 원인 보존율 + marginal gain, 검정 McNemar+BH · 2층: 메모리·TTFT·throughput 실측, 단위당 절감 · 3층: 출력 분포를 log-prob·perplexity·KL로 비교. 세 층이 입력 보존(1)·출력 충실(3)·비용(2)을 함께 본다.